CSE-632-50-4258

Project 2: Trojan Horse Traffic Dataset

Bethanie Bairen

10/22/2025

Project 2: Trojan Horse Traffic Dataset

Abstract: The goal of this project is to understand how data mining can be used in the real-world scenarios such as analyzing if network traffic is benign or malicious and other IT security analysis procedures. The project consisted of a dataset that had to be preprocessed and then used for various data mining models to analyze which model was the best for the most accurate detection in Trojan data. In my findings, I discovered that Random Forest was the best model with a 0.73 accuracy in detecting trojan data as the model can handle this robust dataset.

Problem description. With today's advance in technology cyber threats and malware have become more prevalent. These trojans can be overlooked and deceive users as they gain access to the connected devices exploiting their information. However, there are ways that machine learning can be used to detect these trojan threats.

Review of publications Machine learning models can be used to improve accuracy of the predictions such as using Recurrent Neural Networks (RNN) and Convolutional Neural Networks(CNN) which according to research has achieved over 15% improvement in accuracy of detecting these threats (Samia et al., 2024).

Machine learning techniques are being used in the detection of these trojan threats. As Malware is fast growing it's designed to infect a computer system with a "virus" by stealing information or spying on the user. These trojan viruses are always evolving and there are constantly new needs for virus detection software's to combat them. Which machine learning techniques can be used to help protect the systems as "Machine learning models can be trained to classify the malware into infected execution and legitimate execution" (Shhadat et al., 2020).

Additionally, artificial intelligence is now being used to help with trojan detection. There are more ways that data is being obtained such as video data, social media posts, etc. This increase in the data surface has caused an increase in trojan attacks. AI can used to hack systems but can also be used to protect against attacks like these through anomaly detection by evaluating what the deviation is from the normal baseline. By having these analysis reports, there can be recommended security tools suggested (Budhathoki, 2025).

Software Tools. For Software tools I used Jupyter Notebook and Python scripts to clean up the dataset through preprocessing. Then I used the same tools for creating the Data Mining Models and the heatmap to test the predictions of the dataset.

Preprocessing. When reviewing this data set, I was challenged with how to clean this up to remove problematic or errored data, make the analysis more efficient, and avoid as many false positives or negatives as possible. For my first preprocessing technique. I removed columns that had either blank or irrelevant data this took the columns from the initial 85 columns down to 74 columns. Next in the techniques I got rid of missing or broken data by removing rows with IP addresses that were blank or 0.0.0.0 which took the rows from the initial 159733 down to 159706. Another technique I used was cleaning up the columns to all be cohesive as the column names varied by having spaces in front of the names. Last in techniques, I got rid of data that had infinite values and zeros which took the rows from 159706 down to 140977.

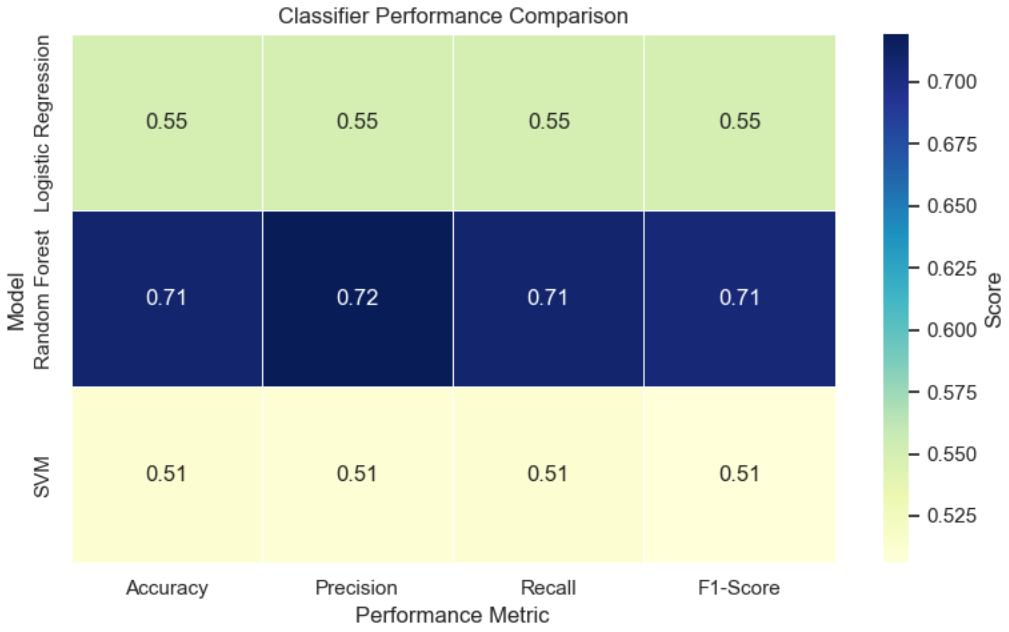
Data mining process. For the data mining techniques, I used Logistic Regression, Random Forest, and Support Vector Machine. I used Logistic Regression as this model is used when the variable has two outcomes, and in this project case the outcomes would be "Trojan" and "Benign". Random Forest models are used when needing multiple decision trees with large datasets with high dimension, this model works for this dataset as the dataset was large and had various branches. Support Vector Machine model is used for classification and regression tasks and trying to find the best hyperplane that differentiates the classes in the data, which in this case would be the "Trojan" and "Benign" data.

When looking at all three models: Logistic Regression, Random Forest, and Support Vector Machine I analyzed the precision, recall, and F1 score for the initial prediction results. For Logistic Regression the precision, recall, and f1 score for benign was 0.54, 0.53, and 0.5. For Trojan it was 0.56, 0.56, and 0.56. Overall, the model provided a 0.55 accuracy which means there is balanced performance between the Benign and Trojan classes.

For Random Forest, the precision, recall, and f1 score for benign was 0.71, 0.75, and 0.73. For Trojan it was 0.75, 0.71, and 0.73. Overall, the model provided 0.73 accuracy which is a higher improvement than the Logistic Regression's 0.55. This means compared to the Logistic Regression model, the Random Forest model caught more in the data that was missed.

Finally for Support Vector Machine Model, the precision, recall, and f1 score for benign was 0.50, 0.40, and 0.44. For Trojan it was 0.52, 0.62, and 0.56. Overall, the model provided a 0.51 accuracy which, compared to Logistic Regression's 0.55 accuracy and Random Forest's 0.73 accuracy, is worse in detecting Trojans which means its missing most of the malicious data. This was the weakest model out of the three and Random Forest was the strongest model.

Results/Conclusion.



When analyzing the models and creating a visualization chart to showcase the differences, I found that Random Forest was the best model in accuracy of catching malicious data. The heat map created shows the three models compared and Random Forest was at the darkest blue of the heat map, then Logistic Regression being somewhat in the lower middle of the heat map with the light green, and Support Vector Machine being last on the heat map with the pale yellow. Random Forest better preformed due to the model's ability to handle non-linear patterns, features, and outliers with use of decision trees. The Logistic Regression model is linear and is unable to separate the Trojan and Benign Data which is why it wasn't the best model for this dataset. Finally, the Support Machine Vector model was not the best model because while it can be non-linear, it is affected by the scaling of

data which means it misclassified some of the data which lowered the accuracy. Thus, Random Forest is the best model for the dataset.

References.

- Budhathoki, N. (2025, August 1). *The role that AI and Machine Learning plays in Cybersecurity*. DSS Blog. https://roundtable.datascience.salon/the-role-that-ai-and-machine-learning-plays-in-cybersecurity?gad_source=1&gad_campaignid=22114556059&gbraid=0AAAAA-Q8n0vt8RnrB4npnd8ayuWY-LOXg&gclid=CjwKCAjwpOfHBhAxEiwAm1SwEmKRFE5p5CqiHwHXCrEK9NH7-xWnaL7Av6Uvv4X0RAdgYNlT6K_yAhoCP_sQAvD_BwE

- Shhadat, I., Bataineh, B., Hayajneh, A., & Al-Sharif, Z. A. (2020). The use of machine learning techniques to advance the detection and classification of unknown malware. *Procedia Computer Science*, *170*, 917–922. https://doi.org/10.1016/j.procs.2020.03.110

- Samia, N., Saha, S., & Haque, A. (2024). Predicting and mitigating cyber threats through Data Mining and machine learning. *Computer Communications*, *228*, 107949. https://doi.org/10.1016/j.comcom.2024.107949

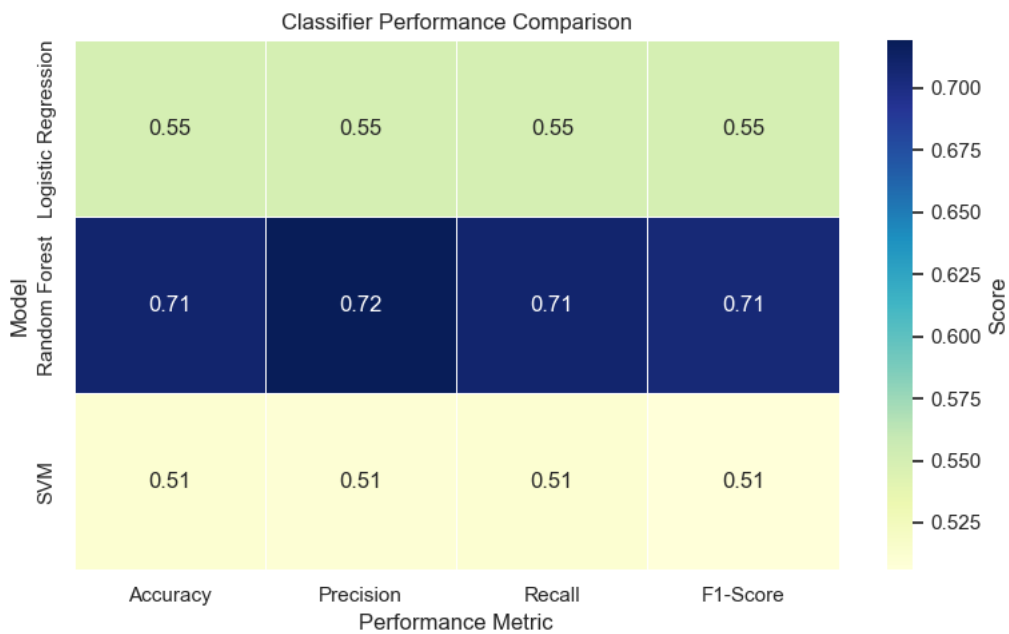
- Merriam-Webster. (n.d.). *Fraud definition & meaning*. Merriam-Webster. https://www.merriam-webster.com/dictionary/fraud

Appendix.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

def load_and_prepare_data(filepath):
    df = pd.read_csv(filepath)

    #Clean column names
    df.columns = df.columns.str.strip().str.replace(' ', '_').str.replace('.', '', regex=False)

    df.drop(columns=['Bwd_Avg_Bytes/Bulk', 'Bwd_Avg_Packets/Bulk', 'Fwd_Avg_Bulk_Rate',
                     'Fwd_Avg_Bytes/Bulk', 'Fwd_Avg_Packets/Bulk', 'ECE_Flag_Count',
                     'CWE_Flag_Count', 'RST_Flag_Count', 'FIN_Flag_Count',
                     'Bwd_URG_Flags', 'Fwd_URG_Flags', 'Bwd_PSH_Flags'], inplace=True, errors='ignore')

    #Drop irrelevant columns
    df.drop(columns=['Unnamed:_0', 'Flow_ID', 'Source_IP', 'Destination_IP', 'Timestamp'], inplace=True,
errors='ignore')

    #Drop duplicate features
    df = df.loc[:, df.nunique() > 1]

    #Replace inf and drop NaNs
    df.replace([np.inf, -np.inf], np.nan, inplace=True)
    df.dropna(inplace=True)

    #Filter invalid IPs and ports
    df = df[df['Source_Port'].notnull() & df['Destination_Port'].notnull()]
    df = df[df['Source_Port'] != '0']
    df = df[df['Destination_Port'] != '0']

    return df
train_df = load_and_prepare_data('Dataset/train_data.csv')
test_df = load_and_prepare_data('Dataset/test_data.csv')

#Training
if 'Class' in train_df.columns:

    X = train_df.drop(['Class'], axis=1)
    y = train_df['Class']

    #Scale features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
```

```
    #Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42, stratify=y
    )

    X_train = np.nan_to_num(X_train)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    #Instantiate models

    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    rf.fit(X_train, y_train)
    y_pred_rf = rf.predict(X_test)

    log_reg = LogisticRegression(max_iter=1000)
    log_reg.fit(X_train, y_train)
    y_pred_lr = log_reg.predict(X_test)

    svm = SVC(kernel='rbf', C=1, gamma='scale', max_iter=1000)
    svm.fit(X_train, y_train)

    #Predict
    y_pred_svm = svm.predict(X_test)

#Testing
if 'Class' not in test_df.columns:
    X_test = test_df.copy()
    X_test_scaled = scaler.transform(X_test)

    #Predict
    preds_lr = log_reg.predict(X_test_scaled)
    preds_rf = rf.predict(X_test_scaled)
    preds_svm = svm.predict(X_test_scaled)

    #Save predictions
    test_df['Pred_LogReg'] = preds_lr
    test_df['Pred_RF'] = preds_rf
    test_df['Pred_SVM'] = preds_svm

    test_df.to_csv('Dataset/test_predictions.csv', index=False)
    print("Predictions saved to test_predictions.csv")
Predictions saved to test_predictions.csv

#Evaluate and compare
def get_metrics(y_true, y_pred, model_name):
    report = classification_report(y_true, y_pred, output_dict=True)
    accuracy = accuracy_score(y_true, y_pred)
    return {
        'Model': model_name,
        'Accuracy': accuracy,
        'Precision': report['weighted avg']['precision'],
        'Recall': report['weighted avg']['recall'],
        'F1-Score': report['weighted avg']['f1-score']
```

```
    }
if 'Class' in train_df.columns:
    metrics_list = []
    metrics_list.append(get_metrics(y_train, log_reg.predict(X_train_scaled), 'Logistic Regression'))
    metrics_list.append(get_metrics(y_train, rf.predict(X_train_scaled), 'Random Forest'))
    metrics_list.append(get_metrics(y_train, svm.predict(X_train_scaled), 'SVM'))

    df_perf = pd.DataFrame(metrics_list)
    df_perf.set_index('Model', inplace=True)

    #Heatmap
    plt.figure(figsize=(8, 5))
    sns.set(style="whitegrid")
    sns.heatmap(df_perf, annot=True, cmap="YlGnBu", fmt=".2f", linewidths=0.5, cbar_kws={'label': 'Score'})
    plt.title("Classifier Performance Comparison")
    plt.ylabel("Model")
    plt.xlabel("Performance Metric")
    plt.tight_layout()
    plt.show()
```




```
from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=20)
X_new = selector.fit_transform(X, y)

selected_features = X.columns[selector.get_support()]
print("Selected Features:", selected_features)
Selected Features: Index(['Flow_Duration', 'Fwd_Packet_Length_Max', 'Fwd_Packet_Length_Std',
       'Flow_IAT_Max', 'Fwd_IAT_Total', 'Fwd_IAT_Std', 'Fwd_IAT_Max',
       'Bwd_IAT_Total', 'Bwd_IAT_Max', 'Bwd_Packets/s', 'Min_Packet_Length',
       'URG_Flag_Count', 'Down/Up_Ratio', 'Active_Mean', 'Active_Max',
       'Active_Min', 'Idle_Mean', 'Idle_Std', 'Idle_Max', 'Idle_Min'],
      dtype='object')
print("Logistic Regression")
print(classification_report(y_test, y_pred_lr))
```

```
Logistic Regression
              precision    recall  f1-score   support

      Benign       0.54      0.53      0.53     13785
      Trojan       0.56      0.56      0.56     14415

    accuracy                           0.55     28200
   macro avg       0.55      0.55      0.55     28200
weighted avg       0.55      0.55      0.55     28200
```

```
print("Random Forest")
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest
              precision    recall  f1-score   support

      Benign       0.71      0.75      0.73     13785
      Trojan       0.75      0.71      0.73     14415

    accuracy                           0.73     28200
   macro avg       0.73      0.73      0.73     28200
weighted avg       0.73      0.73      0.73     28200
```

```
print("Support Vector Machine")
print(classification_report(y_test, y_pred_svm))
```

```
Support Vector Machine
              precision    recall  f1-score   support

      Benign       0.50      0.40      0.44     13785
      Trojan       0.52      0.62      0.56     14415

    accuracy                           0.51     28200
   macro avg       0.51      0.51      0.50     28200
weighted avg       0.51      0.51      0.50     28200
```